



Model-based decision support for knowledge-intensive processes

Anjo Seidel¹ · Stephan Haarmann¹ · Mathias Weske¹

Received: 9 March 2022 / Revised: 30 August 2022 / Accepted: 30 November 2022 /
Published online: 20 December 2022
© The Author(s) 2022

Abstract

Process-aware information systems guide participants through the execution of processes. However, existing systems have limited support for knowledge-intensive processes, which are multi-variant and shaped by informed decisions of knowledge workers. Yet, making such decisions causes high cognitive load, as the effect of the decision on the future process execution must be considered. This may cause errors and/or slow down the process execution. We present an approach based on fragment-based Case Management. It supports the iterative decision making by (i) enabling knowledge workers to define goals and (ii) by giving recommendations on which decision outcomes align with the goals and which do not. For that, we use information from the process model and the running process instance. We show the technical feasibility with a proof-of-concept implementation and the value for knowledge workers in a preliminary user study.

Keywords Case management · Decision support · Process modeling · Model checking · Fragment-based case management

1 Introduction

In the modern information economy, knowledge work is the key activity associated with economic growth (Elliott & Jacobson, 2002). While different definitions of knowledge work exist, knowledge workers generally use their expertise and experience to identify and

✉ Anjo Seidel
anjo.seidel@student.hpi.de

Stephan Haarmann
stephan.haarmann@hpi.de

Mathias Weske
mathias.weske@hpi.de

¹ Hasso Plattner Institute, University of Potsdam,
Prof.-Dr.-Helmert Str. 2–3, Potsdam, 14482, Germany

solve problems (Pyöriä, 2005). Thereby, knowledge workers make informed decisions, from which a solution emerges gradually (Ciccio et al., 2015).

Physicians are example knowledge workers. To treat patients, physicians gather information, for example, during anamnesis, examinations, and blood tests. Physicians decide on a treatment based on this data, their knowledge, and their experience. Yet, new knowledge—such as an adverse drug reaction—is gathered and may be used to adjust the treatment. Knowledge work also exists in other domains, e.g., during the claim handling at insurance companies.

Processes that are primarily concerned with knowledge work are called *knowledge-intensive processes*. They are data-centric, goal-driven, and multi-variant (Ciccio et al., 2015). *Case management* is a process management paradigm designed for such processes (Swenson, 2013). A case is concerned with a subject (e.g., the patient) in a situation (e.g., the disease), and the goal of the case is the resolution of the situation (e.g., the successful treatment) (Object Management Group, 2016). A challenge is that knowledge is often volatile and vast, so knowledge-intensive processes are unrepeatable and unpredictable (Ciccio et al., 2015; Swenson, 2013). Respective process models are often under-specified and may be adapted at run-time (Ciccio et al., 2015). This makes case management challenging.

Information systems can be used to gather and organize data. Process-aware information systems may also determine enabled activities, which are compliant with a process model (Weske, 2019). But, in knowledge-intensive processes, there may be many enabled activities (Ciccio et al., 2015). Information systems, in general, cannot determine which activities align with the goals of the knowledge worker. Instead, the worker has to be aware of their goals, enabled activities, and their relationship. This causes cognitive load and may lead to errors.

This gap leads to our research questions:

RQ1 How can knowledge workers' goals be specified using the explicit information available in process models and cases?

RQ2 How can we determine enabled activities that align with modeled goals?

By determining activities that support a modeled goal, knowledge workers receive decision support: Activities that contradict a goal do not need to be considered during a decision.

Our contribution includes conceptual solutions for RQ1 and RQ2. Based on the *fragment-based Case Management (fCM)* approach (Hewelt & Weske, 2016), we conceptualize case models and cases to specify respective goals using *first-order logic (RQ1)*. We translate the goals into a set of model checking problems to determine compatible activities (RQ2). This paper is an extension of our previous publication (Haarmann et al., 2021) proposing a method for modeling goals w.r.t. an fCM model.

This article is structured as follows: First, we give necessary background information regarding knowledge-intensive processes, case management, and model checking (Section 2). Afterward, we present related work (Section 3). Section 4 details the specification of goals. In Section 5, we describe the translation of goals to model checking problems. We evaluate our contribution with a proof of concept implementation, which we use for performance measurements and a user study (Section 6). Finally, we conclude our work (Section 7).

2 Background

In this paper, we combine different lines of research. On the one hand, our approach is based on modeled knowledge-intensive processes. We furthermore employ model checking to provide decision support for knowledge workers. In this section, we present respective preliminaries.

2.1 Knowledge-intensive processes

Data is the fuel for the information economy, and the demand for people making good business decisions by gathering, accessing, and making sense of data is increasing (Elliott & Jacobson, 2002). These people are called knowledge workers (Pyöriä, 2005). They use their expertise and experience for tasks, such as problem identification, problem-solving, and planning (Pyöriä, 2005). They learn continuously to keep their knowledge up-to-date (Pyöriä, 2005), and they need to cope with novel and unique situations (Ciccio et al., 2015).

In business process management, activities conducted in an enterprise are organized into processes (Weske, 2019). Processes that focus on knowledge work are called knowledge-intensive. They differ from traditional processes (Swenson, 2012), which becomes evident in the list of characteristics elicited by Di Ciccio et al., (2015). Of course, knowledge-intensive processes are *knowledge-driven*: Knowledge workers consider available information to choose a course of action, but this knowledge cannot be captured holistically and efficiently. Therefore, accurate *predictions* are impossible. Instead, processes *emerge* gradually with each decision, which knowledge workers align with their *goals*. Furthermore, two instances of a knowledge-intensive process always differ—the process is *non-repeatable*.

Traditional processes, on the other hand, are usually limited to a few well-defined variants with clearly defined decision rules. Respective process management paradigms are therefore not well suited for knowledge-intensive processes (Swenson, 2013). As an alternative, case management was proposed.

2.2 Fragment-based Case Management

The case management paradigm addresses the needs of flexible data-centric processes. A case has a subject that is in a situation, which needs to be resolved (Object Management Group, 2016). In a hospital setting, the subject is a patient, the situation is a disease, and the resolution is a successful treatment. The knowledge workers are responsible for choosing and executing actions that change the situation and eventually resolve it. Case management supports the collection, organization, and management of relevant information. Furthermore, it asserts that knowledge workers adhere to essential constraints.

Multiple case management approaches exist (Steinau et al., 2019). Cases are described by models, which commonly consist of a data model structuring the information about cases and a behavioral model defining activities and behavioral constraints. These constraints are often expressed declaratively to remain concise in the presence of flexibility (van der Aalst et al., 2009).

Our contribution builds upon the fragment-based Case Management approach (fCM) (Hewelt & Weske, 2016). Since many knowledge-intensive processes contain structured and unstructured parts, fCM models include structured, control flow-based fragments that can be composed dynamically within data constraints (Meyer et al., 2014). Similar to BPMN (Object Management Group, 2014), data is organized in data objects that

have a finite set of abstract states (Hewelt & Weske, 2016). In contrast to BPMN, links among objects (Haarmann & Weske, 2020) and data associations with multiplicity constraints (Haarmann et al., 2021) play an important role.

2.2.1 Fragments

At the core of every fCM model are fragments (Hewelt & Weske, 2016). These are acyclic control flow graphs, similar to BPMN processes. They contain start events, activities, exclusive gateways, as well as data object nodes. A start event denotes the beginning of a new case. Fragments without a start event can therefore be started while the case is running. These fragments can run repeatedly, and fragment instances can be executed concurrently.

Data object nodes play a crucial role (Haarmann et al., 2021). They are grouped into input and output sets. When an activity is executed, it reads objects for one input set and writes objects for one output set. Subsequently, input sets represent data requirements. Activities can only be executed if the respective data requirements are satisfied. Therefore, data constraints the behavior.

In the following, we describe the fragments for the claim handling process of an insurance company. Fragment F1 in Fig. 1 shows the happy paths of the process: a claim is received. Its risk is assessed. It is rejected or approved in a decision, and it is disbursed, or a rejection letter is sent subsequently.

In some cases, a decision cannot be made directly. The claim remains in question and one or multiple expert assessments are requested (F2) and created and reviewed (F3) as depicted in Fig. 2. During the review, an assessment may be rejected if it does not satisfy formal requirements. In this case, it can be improved by re-executing fragment F3.

Given a set of approved assessments, a claim can be reassessed, creating an advice object (F4) (see Fig. 3). If more assessments are requested, the claim may be reassessed again. Eventually, the decision is revised (F5): it can be approved, rejected, or remain in question.

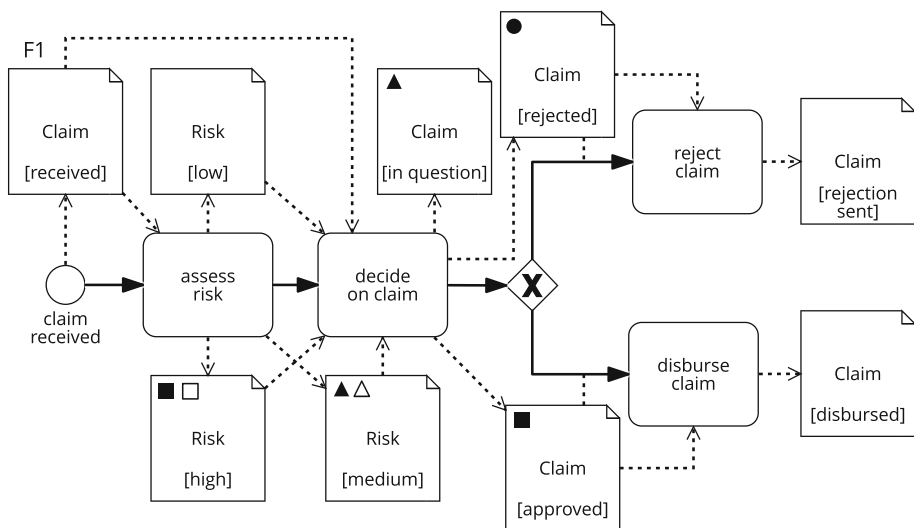


Fig. 1 Fragment 1 for handling insurance claims

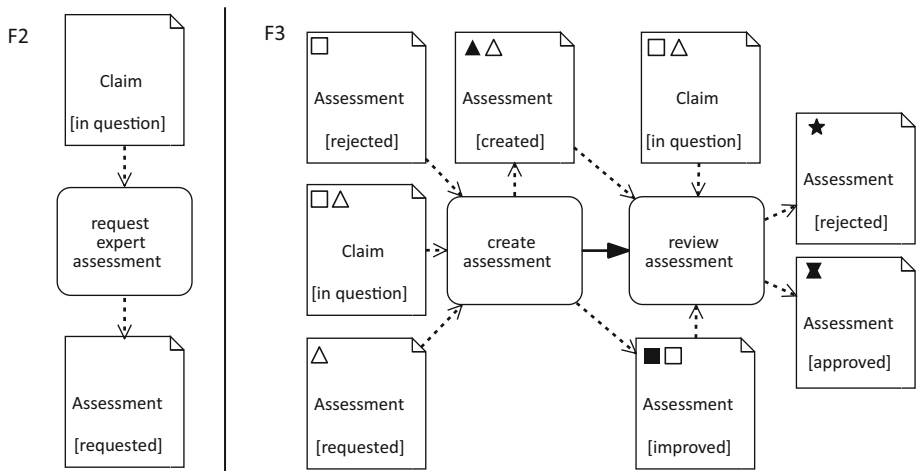


Fig. 2 Fragments 2 and 3 for handling insurance claims

2.2.2 Domain model

Data-centric process modeling approaches commonly include a data model (Steinau et al., 2019). It defines the structure of data objects as well as the possible relationships among objects. In fCM, the case model-specific data model is called domain model (Hewelt & Weske, 2016). It is a class diagram consisting of classes, binary associations, and multiplicity constraints—similar to UML class diagrams (Object Management Group, 2017).

Furthermore, fCM defines additional constraints that every domain model must satisfy (Haarmann et al., 2021): Two classes are connected by at most one association. All associations are existential—this means at least one of the corresponding multiplicity constraints has a positive upper bound. These constraints are necessary to define clearly when

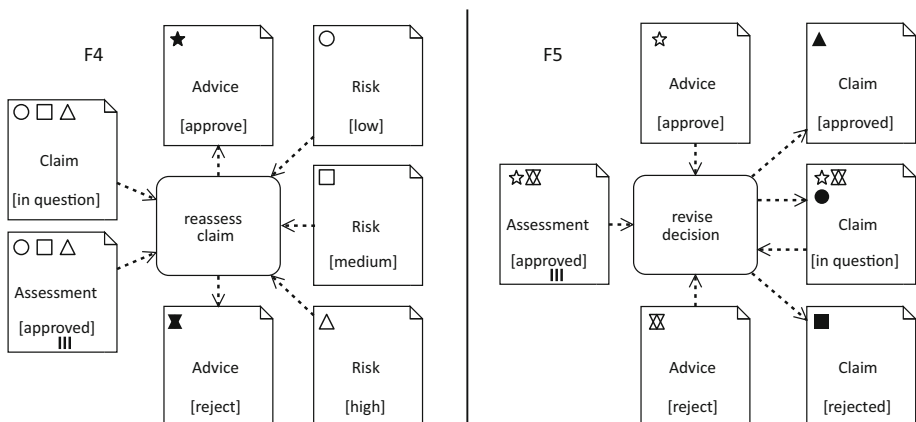


Fig. 3 Fragments 4 and 5 for handling insurance claims

two objects get linked (Haarmann et al., 2021), and similar constraints exist in other data-centric process modeling approaches (Snoeck, 2014; Snoeck et al., 2021).

The domain model for the fragments is shown in Fig. 4. It consists of one class for each type of data objects: claim, risk, assessment, and advice. Furthermore, associations model the relationship among corresponding objects. Each claim is associated with up to one risk, five assignments, and five advice objects. Furthermore, each advice is based on at least one assignment.

2.2.3 Object behavior

Each data object has a set of attributes and a set of links. Jointly, they define the state of the data object. However, in business process management, data object states are often limited to a finite set of states. This also holds for fCM: data object nodes in fragments refer to data objects of a specific class in a specific state (Hewelt & Weske, 2016). Activities may update the state of objects. All possible updates are called the object behavior (Haarmann et al., 2021).

In fCM, object behaviors are described by state transition systems. A case model has a state transition for each class. It defines all possible states and state transitions of corresponding objects. States are an abstraction for the possibly infinite many data states a data object can be in. The level of abstraction for those states is up to the modeler.

The object behavior for the insurance claim handling process consists of one state transition system for each class (see Fig. 5). The claim may be in one of the states *received*, *in question*, *approved*, *disbursed*, *rejected*, and *rejection sent*, and it may transition from one state to another as depicted in Fig. 5. However, object behavior models are not necessarily connected graphs. A risk, for example, may be high, medium, or low, but its state cannot change.

2.2.4 Termination condition

The termination condition defines the desired outcome of the cases. It is a logical expression that is evaluated on the data state of the case: data object nodes act as propositions. They are connected using the logical operators AND ($\wedge$) and OR ($\vee$). Without loss of generality, we assume that the termination condition is in disjunctive normal form: it is a disjunction of product terms.

Similar to an input set, a product term is satisfied if a set of linked objects that adheres to the specified constraints exists. The complete expression is satisfied if at least one product term holds. If both the termination condition holds, the case can be closed.

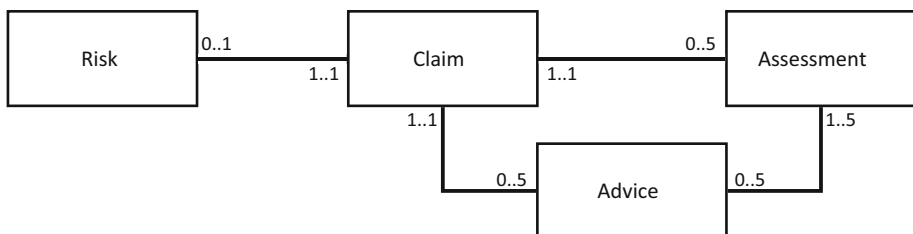


Fig. 4 Domain model for handling insurance claims

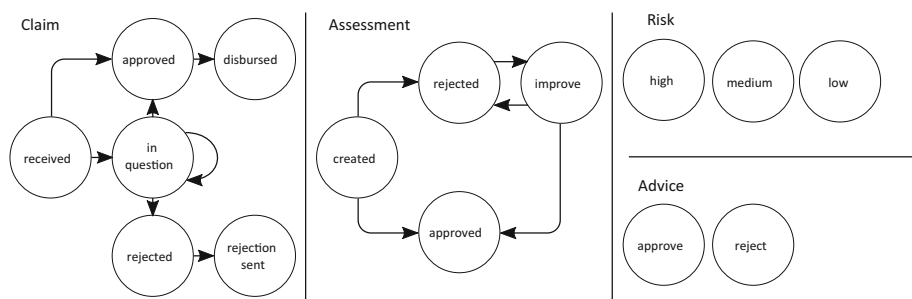


Fig. 5 Object behaviors for handling insurance claims

The termination condition of the example requires the claim to be in state *rejection sent* or *disbursed*. This is expressed by the following condition:

$$Claim[rejection\ sent] \vee Claim[disbursed]$$

2.2.5 Case model

A case model consists of a set of fragments, a domain model, a set of object behaviors that are assigned to classes in the domain model, and a termination condition. The fragments define the activities, their dependencies, data requirements, and data operations. The domain model constrains the structure of the case data. The object behaviors limit the states and state transitions per class, and the termination condition specifies the case goal. To provide more flexibility and the means to adapt a case to environmental changes, fCM allows adapting the the case model and its components during run-time.

These relationships among the elements of the case model can be observed in the insurance example. The fragments allow arbitrarily many assessments, but the domain model limits the number of assignments to five per claim.

Activity ‘create assessment’ (see Fig. 2) has two input sets and two output sets: it reads a claim in state *in question* and an assessment either in state *rejected* or *requested*. It writes the assessment in state *created* or *improved*. However, according to the assessment’s behavior, it cannot change from state *requested* to *improved* nor from state *rejected* to state *created*. Therefore, ‘create assessment’ has only two valid input-output-set combinations.

Due to the relationships among the components of the case model, consistency is important. A case model must satisfy the following properties:

Object Behavior Conformance Activities must comply with the object behaviors. They may only perform state changes modeled in the transition systems.

Contextual Object Creation When an activity creates an object, it must read or co-create all objects required to create the novel object. The objects which are required depend on the associations and multiplicity constraints.

Contextual Batch Processing When a set of similar objects is read, a reference object must be read to determine the set unambiguously. The set consists of all objects of the specified class that are linked to the reference object.

The metamodel in Fig. 6 provides an overview of fCM case models. The domain model, object behaviors, fragments, and the termination condition are connected, showing clearly the data-centric nature of fCM.

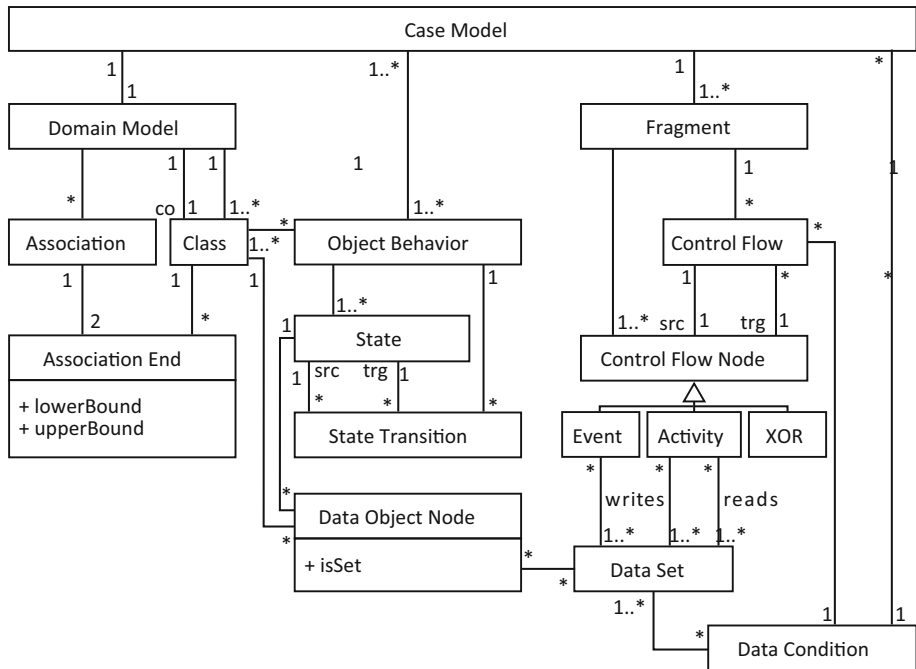


Fig. 6 Metamodel for fCM case models (adapted from Haarmann,(2021))

2.2.6 Execution semantics

Fragments can be executed dynamically and repeatedly according to the control- and data-flow as long as the constraints of the domain model and the object behavior are not violated. The semantics combine the different elements of a case model. Therefore, a mapping of fCM case models to colored Petri nets has been defined (Haarmann et al., 2021). It asserts that activities are executed without violating the object behaviors or the domain model.

During a case, activity instances are executed, instantiating classes and associations and updating data objects. Furthermore, control flow is advanced. We call an activity instance that is reading and writing specific data objects an action. Actions can be in one of the following states: *init*, *control flow enabled*, *data-flow enabled*, *enabled*, *running*, or *terminated* (cf. Fig. 7).

The state of a case is determined by the data objects, the links among them, and the actions as defined in Definition 1.

Definition 1 (Case State) Given a case model, let $\mathcal{A}$ be the set of activities, $\mathcal{C}$ the set of classes, $\mathcal{Q}_c$ be the set of states for a class $c \in \mathcal{C}$. Furthermore, let $\mathcal{ID}$ be the set of possible identities. The state s of a case is defined by a tuple $s = (O, L, A)$.

- O is a set of data objects, where each data object $o \in O$
 - is the instance of the class $o.class \in \mathcal{C}$.
 - has a state $o.state \in \mathcal{Q}_{o.class}$ and the identity $o.id \in \mathcal{ID}$.

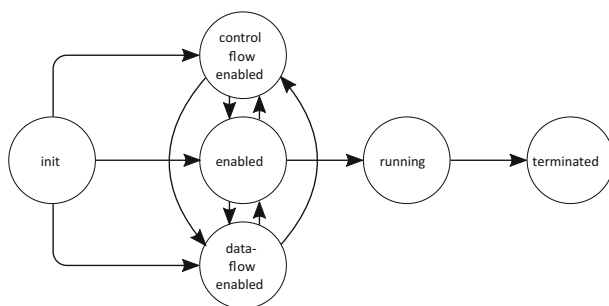


Fig. 7 Life cycle of an activity instance in fCM

- $L \subseteq \{\{o_1, o_2\} | o_1, o_2 \in O\}$ is a set of links, where each link is an unordered pair of objects.
- A is a set of actions, where each action $a \in A$
 - is an instance of activity $a.activity \in \mathcal{A}$.
 - is in state $a.state \in Q_{\mathcal{A}}$, where $Q_{\mathcal{A}} = \{init, df-enabled, cf-enabled, enabled, running, terminated\}$.
 - reads a set of data objects $a.reads \subseteq O$.
 - writes a set data objects $a.writes$.¹

We use the case state for modeling goals and colored Petri net formalizations of case models (cf. (Haarmann et al., 2021)) for model checking-based recommendations.

2.3 Model checking

Model checking is a formal method to prove whether a behavioral model satisfies given properties (Clarke et al., 2001; Baier & Katoen, 2008): the model and the properties are formally defined. A model checker takes both, generates the model’s state space, and searches it to find paths that violate or satisfy the property. In business process management, model checking is used to check generic properties, such as soundness (van der Aalst et al., 2011), and domain-specific ones, such as compliance rules (Knuplesch & Reichert, 2017; Awad et al., 2011).

We use model checking for decision support. We formalize fCM case models with colored Petri nets as described in (Haarmann et al., 2021). We formalize properties with computational tree logic (Clarke & Emerson, 1981). A model checker (i.e., CPN Tools)² proves whether the property can be satisfied and may provide a *witnesspath* or *counterexample*.

Computational tree logic (CTL) (Clarke & Emerson, 1981) extends propositional logic with *temporal operators* and *path quantifiers*. It can express statements about state transition systems, where each state has a set of propositions. Thereby, we can express statements, such as “In all paths, the case will terminate eventually”.

¹ $a.writes$ may contain data objects not in O , i.e., objects that are created or updated.

²<https://cpntools.org/>

Formally, both universal and existential path quantifiers exist in CTL. **A** requires that the formula holds for all paths and **E** that the formula holds for at least one path. Furthermore, the following *temporal operators* are available:

X ϕ Formula ϕ must hold in the next state.

F ϕ Formula ϕ must hold eventually.

G ϕ Formula ϕ must always hold.

$\phi_1 \mathbf{U} \phi_2$ Formula ϕ_1 holds until ϕ_2 holds, and ϕ_2 must hold eventually.

In CTL, path quantifiers and temporal operators always occur in pairs: a path quantifier is always followed by a temporal operator, and a temporal operator is always preceded by a path quantifier. Consider the natural language example from above. Let *terminated* be the propositional variable which is true if the case has terminated. In CTL, the statement above is formalized as follows:

$$\mathbf{AF}(\textit{terminated})$$

Due to space limitations, we limit the discussion of model checking to the fundamental idea. Details, e.g., how a model checker verifies a model against a CTL formula, can be found in the respective literature (Clarke et al., 2001; Baier & Katoen, 2008).

3 Related work

In this section, we provide an overview of related work. We address knowledge intensive-processes, modeling of knowledge-intensive processes, and decision support and planning in BPM.

According to (Pyöriä, 2005), many definitions of knowledge work exist. We focus on the work of experts who are concerned with identifying and solving problems, and who gather and assess data to make informed decisions about their next actions. Their work is organized into knowledge-intensive processes (Ciccio et al., 2015).

Knowledge-intensive processes are often under-specified since knowledge workers' expertise and experience are vast and fast-changing (Swenson, 2013). Ciccio et al. (2015) characterize knowledge-intensive processes and deduce a set of requirements. Among other things, such processes are unrepeatable and unpredictable and therefore require flexibility at design- and run-time.

Traditional process modeling languages, such as BPMN (Object Management Group, 2014), define process variants explicitly and are therefore considered unsuited for knowledge-intensive process (Swenson, 2012). To address this challenge, the research community has proposed novel process modeling approaches. Two trends are apparent: process models become more (i) declarative and (ii) data-centric (Ciccio et al., 2015).

The two most widely used purely declarative approaches are DECLARE (Pesic et al., 2007) and DCR-Graphs (Hildebrandt & Mukkamala, 2010). Rather than defining variants explicitly, declarative approaches define ordering constraints among activities. Every instance that does not violate these constraints is valid. DECLARE and DCR-Graphs use constraints based on linear temporal logic over finite traces.

Data-centric approaches, on the other hand, focus on data requirements, operations, and conditions (Steinau et al., 2019). The Guard-Stage-Milestone approach, for example, defines stages with pre- and post-conditions (Hull et al., 2010). The possible order of stages is given declaratively by these conditions. Other data-centric approaches, such as PHILharmonicFlows (Künzle & Reichert, 2011) and BAUML (Estañol, 2017), are more

procedural: they define imperative models for each data type and allow composing them into larger processes. A recent overview of data-centric approaches is provided by Steinau et al. (2019). Hybrid approaches aim to combine the advantages of data-centric and declarative approaches. OCBC (van der Aalst et al., 2017) provides a combined syntax for both model types, while RESEDA (Seco et al., 2018) also incorporates event. With ReGraDa (Galrinho et al., 2021) a novel process definition and programming language was provided to execute processes on a database level. Case management approaches also address the requirements of knowledge-intensive processes (Swenson, 2013). They are data-driven, flexible, and often adaptable, i.e., the model can change at run-time. The fragment-based Case Management approach (Hewelt & Weske, 2016; Haarmann et al., 2021) is considered hybrid, since it combines activity-centric, imperative fragments and declarative data conditions. The highly structured parts (the fragments) can be combined dynamically if it is supported by the data dependencies. Furthermore, fCM is adaptable because new fragments can be added at run-time.

While these languages are all capable of modeling aspects of knowledge-intensive processes, they offer limited guidance at run-time. Though planning is crucial for knowledge work (Pyöriä, 2005), only a few approaches support late goal modeling (Ciccio et al., 2015), i.e. the modeling during run-time of goals to reach via planning.

In BPM, different approaches focus on automated planning. Marrella et al., (2014) present SmartPM where planning can be used to handle exceptions such that the data state can be automatically corrected. Venero et al., (2020) proposed planning for knowledge-intensive processes utilizing Markov decision processes. Furthermore, Sprovieri and Vogler (2015) apply planning on partly structured models to compose processes. However, as knowledge-intensive processes are emergent and driven by knowledge workers, automated planning is not fully applicable. Other approaches aim to provide decision support for knowledge workers during the execution of a process. Wynn et al., (2007) and Rozinat et al., (2009) offer decision support by simulation of business processes. Also, various contributions for predictive process monitoring exist, which predict the next actions in an ongoing process with machine learning techniques. The surveys of Teinemaa et al., (2019) and Francescomarino et al., (2018) provide an overview. Unfortunately, knowledge-intensive processes are characterized as non-repeatable and non-predictable (Ciccio et al., 2015). Therefore, simulations cannot necessarily be repeated, and machine learning models might not be applicable.

4 Modeling goal constraints

We aim at providing decision support for knowledge workers w.r.t their goals. In our model-based approach, a goal consists of constraints on case states. We consider a goal to be a combination of multiple objectives. To satisfy the goal, all corresponding objectives need to be reached. An objective is an operative goal. It is a statement about a single execution state, i.e., actions as well as data objects, their states, and links. In this section, we first define objectives, before we discuss temporally combining multiple objectives into goals.

4.1 Objectives

An objective is an operative goal and considered with a single case state. It can be formally defined as a logic expression over any case state (Definition 2). If it evaluates to true for a given state, the objective is satisfied.

Definition 2 (Objective) Given a case and its case model, let $s = (O, L, A)$ be a state of the case, O the set of data objects, L the set of links between them, and A the actions. G is the set of possible objectives over the state s and the case model. A data objective $g \in G$ is a first-order logic expression about the sets O , L , and A .

Objectives are required to be within the scope of the case model. They can make statements about the actions in an execution state and the data state of a case. The actions are instances of the modeled activities and have a state according to their life cycle. The data state consists of a set of linked data objects. Each such object has a state that is modeled in the fragments and object behavior. It abstracts from the concrete values of the attributes. To be within the scope of a model, an objective can make statements about object states but not attributes.

Objectives may make statements about the actions A , the data objects O with their classes, object states, and unique IDs, and the links of data objects L in a certain execution state. They can be used to formulate the following objectives:

g_1 : The knowledge worker wants the advice with ID *advice1* linked to two approved assessments.

$$g_1 \equiv \exists o_1 \in O : | \{ o_2 \in O \mid o_2.class = Assessment \wedge o_2.state = approved \wedge \{ o_1, o_2 \} \in L \} | = 2$$

g_2 : The knowledge worker wants to be able to execute the activity ‘revise decision.’

$$g_2 \equiv \exists a \in A : a.activity = revise\ decision \wedge a.state = enabled$$

g_3 : The knowledge worker wants to execute ‘revise decision’ reading an advice object that is linked to all existing assessments.

$$g_3 \equiv \exists a \in A, o_1 \in a.reads, \forall o_2 \in O : a.activity = revisedecision \wedge a.state = enabled \wedge o_1.class = Advice \wedge (o_2.class = Assessment \implies \{ o_1, o_2 \} \in L)$$

4.2 Combined goal constraints

A goal is a temporal combination of objectives. To satisfy a goal, the objectives need to be satisfied in an execution sequence in a given order. The temporal order of objectives can be defined as follows:

Definition 3 (Temporally Ordered Objectives) Let $p = (p_s, p_a)$ be an execution sequence with the sequence of states $p_s = \langle s_1, s_2, \dots, s_n \rangle$ and the sequence of actions $p_a = \langle a_1, a_2, \dots, a_{n-1} \rangle$. Two objectives g_1 and g_2 are temporally ordered $g_1 \leq_t g_2$ in p if g_1 is satisfied in a state $s_i \in p_s$, g_2 is satisfied in a state $s_j \in p_s$, and $i \leq j$.

Definition 4 (Goal) Let G be a set of objectives and $\leq_t$ the temporal order relation for objectives. Γ is the set of goals, which are temporal order relations for the objectives G . A goal $\gamma \in \Gamma$ is a partial order $\gamma = (G', \leq_t)$ for a set of objectives $G' \subseteq G$.

A goal (Definition 4) is a partial order of objectives for the temporal order relation $\leq_t$. Consider the following example of a goal:

γ_1 : First, the knowledge worker wants to gather three assessments in state *approved*. Afterward, they want to create a single advice object. Once both the set of assessments and

the advice object exist, they want to execute ‘revise decision’. Formally, we break the goal into three objectives:

$$\begin{aligned} g_{4a} &\equiv |\{o \in O \mid o.class = Assessment \wedge o.state = approved\}| = 3 \wedge \\ &\quad |\{o \in O \mid o.class = Advice\}| = 0 \\ g_{4b} &\equiv |\{o \in O \mid o.class = Advice\}| = 1 \\ g_{4c} &\equiv \exists a \in A : a.activity = reviseddecision \wedge a.state = enabled \end{aligned}$$

The objectives are ordered in a sequence $\langle g_{8a}, g_{8b}, g_{8c} \rangle$. This results in the following formalization of the combined goal γ_1 :

$$\gamma_1 = (\{g_{4a}, g_{4b}, g_{4c}\}, \{(g_{4a}, g_{4a}), (g_{4b}, g_{4b}), (g_{4c}, g_{4c}), (g_{4a}, g_{4b}), (g_{4a}, g_{4c}), (g_{4b}, g_{4c})\})$$

5 Decision support

Modeling goals is only the first step to providing decision support for knowledge workers. We use the goals to inform knowledge workers about enabled actions that are compatible with their goals. For this purpose, we use model checking.

Our approach (see Fig. 8) requires the following inputs: a formalization of the case model, i.e., its colored Petri net representation (cf. Haarmann et al., 2021), the current state of the case, and the goal of the knowledge worker. Based on these inputs, we generate a set of model checking problems. For each action that is enabled in the current state, we check whether it is compatible with the goal. This information is provided to the knowledge worker.

5.1 Translating goals into model checking problems

First, we consider objectives, which are evaluated on one state. Given the set of actions, objects, and links, the first-order logic formula of the objective is true or false. If it is true, reaching the state implies accomplishing the objective. We assume that knowledge workers

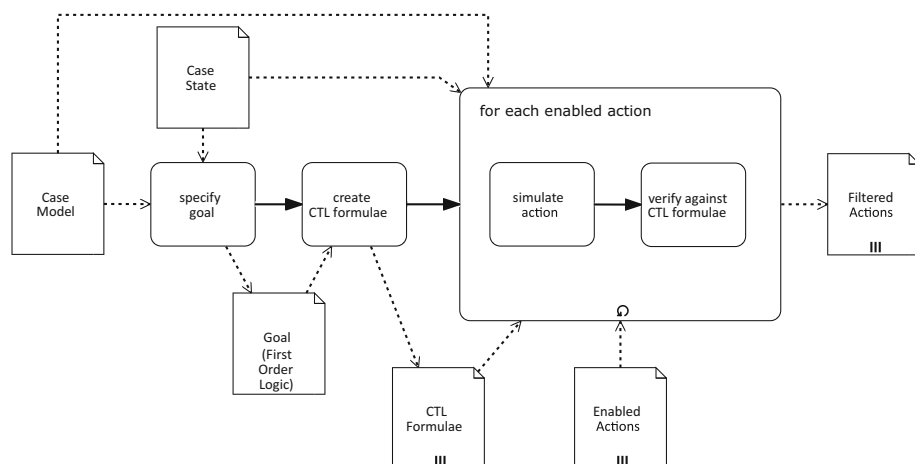


Fig. 8 Approach for filtering enabled actions based on a given objective

are only interested in reaching objectives. They might become obsolete over time and within one goal.

To check whether an action is compatible with an objective, we simulate the action to retrieve the corresponding successor state. Then, we use a model checker to check if we can reach a state that satisfies the objective. Therefore, we translate the objective into a temporal logic formula. Let g be an objective of the knowledge worker, it is transformed into the following CTL formula: $\mathbf{EF}(g)$. It reads ‘In the current state starts a path with a state satisfying the objective.’ We allow goals to combine different objectives. Objectives might become obsolete over time. Therefore, we are only interested in reaching objectives, denoted by $\mathbf{EF}(g)$. They do not need to hold globally.

Given the example objective g_2 , ‘The knowledge worker wants to have an advice object based on all assessments’, the knowledge worker wants to advance the case to a state, in which an advice object exists, and this advice object is linked to all assessments. With the CTL formula $\mathbf{EF}(g_2)$, we can verify that the objectives can be reached from the current state.

CTL is a propositional logic (Clarke & Emerson, 1981), objectives are specified in first-order logic. To solve this mismatch, we create a propositional variable for each objective. For each state, we check whether it satisfies the objective. If so, the respective propositional variable is true, otherwise false. Depending on the implementation of the model checker, it is possible to either compute the value of the propositional variables for each state in advance, or to evaluate the propositional variables ad hoc during the execution of the query.

Given a goal (a partial order of objectives), we derive all strictly ordered sequences of objectives. Each sequence $\langle g_1, g_2, \dots, g_n \rangle$ is translated into CTL:

$$\mathbf{EF}(g_1 \wedge \mathbf{EF}(g_2 \wedge \dots \wedge \mathbf{EF}(g_n) \dots))$$

Given multiple sequences translated into terms $t_1, t_2, \dots, t_m$, we combine them into one formula: $t_1 \vee t_2 \vee \dots \vee t_m$

Consider the following example: the knowledge worker wants to create at least three assessments and later terminate the case. The propositional variables are defined as follows:

$$\begin{aligned} g_{9a} &\equiv |\{o \in O \mid o.class = Assessment\}| \geq 3 \\ g_{9b} &\equiv terminated \end{aligned}$$

This results in the following CTL formula: $\mathbf{EF}(g_{9a} \wedge \mathbf{EF}(g_{9b}))$

5.2 Filtering enabled actions

Given the CTL representation of the goal, we can filter the set of enabled actions. For each such action, we derive the corresponding successor state and use a model checker to verify if the successor state is compatible with the goal. If so, the action may be taken by the knowledge worker. Of course, additional external information needs to be considered. If no suitable action exists, knowledge workers might adapt their goal or the model so that an existing or an alternative action allows reaching the goal. Either way, the feedback of the model checker is provided to assist knowledge workers during their decision making.

Consider the following example: A knowledge worker at our example insurance company assesses the risk of a claim to be high. Next, they decide that assessments are necessary, i.e., the claim is in state *in question*. The knowledge worker plans to request at least four

assessments, wait for their creation before deriving an advice to reassess the claim. They express this plan as a goal:

$$\begin{aligned} g_{10a} &\equiv |\{o \in O \mid o.class = Assessment\}| \geq 4 \wedge |\{o \in O \mid o.class = Advice\}| = 0 \\ g_{10b} &\equiv |\{o \in O \mid o.class = Advice\}| = 1 \\ g_{10} &\equiv (\{g_{10a}, g_{10b}\}, \{(g_{10a}, g_{10a}), (g_{10a}, g_{10b}), (g_{10b}, g_{10b})\}) \end{aligned}$$

Using our approach, the knowledge worker can easily avoid actions that make the goal unreachable. Therefore, the goal g_{10} is translated into a CTL formula:

$$\mathbf{EF}(g_{10a} \wedge \mathbf{EF}(g_{10b}))$$

Consider the state, where the claim is in question, the risk is low, and one assessment has been requested and approved. Activities ‘request expert assessment’ and ‘reassess claim’ are enabled. Both are simulated, and the resulting states are verified against the CTL formula. The state after executing ‘reassess claim’ violates the goal. Executing ‘request expert assessment’ is compatible with the goal. Thus, the knowledge worker should execute the latter.

Knowledge-intensive processes emerge, and knowledge workers need to make decisions frequently to drive the process toward their goals (Ciccio et al., 2015). Using our approach iteratively, they can model their current goal and receive recommendations for the current situation, leading them toward their goals.

6 Evaluation

To evaluate our approach, we first provide a proof of concept implementation, which supports modeling goals using forms. It translates the modeled goal into a model checking problem. Second, we evaluate the technical feasibility of our approach. Therefore, we measure the performance using the model checker provided by CPN Tools³. Finally, we conduct a preliminary user study to show that our approach offers valuable decision support for knowledge workers.

6.1 Proof of concept implementation

Our implementation has two aims. First, the knowledge worker must be allowed to specify goals according to the presented framework. Second, recommendations according to this goal need to be derived from the case. This allows knowledge workers to be assisted in making decisions towards their goals.

The prototype consists of a combination of tools (see Fig. 9). The *fcm2cpn*⁴ translator allows translating an fCM model into its formal representation as a colored Petri net (CPN). This CPN can be executed by a knowledge worker using the CPN modeling and simulation tool *CPN Tools*.

During the execution of an fCM, the *fCM-query-generator*⁵ can be used to define goals at run-time. It produces a state space query that can be used in CPN Tools to analyze the state space of the case. The query can determine if a certain state lies on a path that leads to a state satisfying the goal.

³<http://cpntools.org>

⁴Source and documentation: <https://github.com/bptlab/fcm2cpn>

⁵Source and documentation: <https://github.com/bptlab/fCM-query-generator>

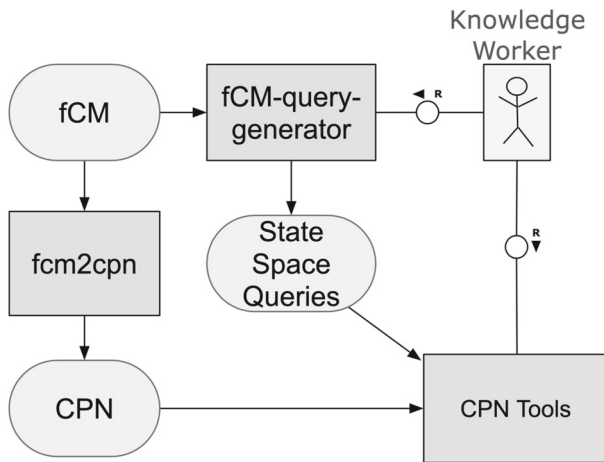


Fig. 9 The architecture of involved components for the prototype

In the query generator, the knowledge worker can select an fCM case model. The generator parses the model and generates a form for specifying a goal using the objects and activities of the case model. The knowledge worker can define statements consisting of data goals and activity goals. Data goals are existential or universal qualifications or multiplicity constraints about data objects and their states. Activity goals describe activities that should be enabled. A number of data and activity goals can be concatenated by using AND, OR, and NOT.

Considering the case model for a claim handling process, Fig. 10 depicts the input form for the goal of having the activity ‘disburse claim’ enabled, having exactly one approving advice, and at least 4 approved assessments.

The goal is stored as a logical expression, and a state space query is generated. This query is a temporal logic expression in the CTL derivative ASK-CTL.⁶ It evaluates the objectives for each state ad hoc and checks whether the goal can be reached from a given state.

To receive decision support during the execution in CPN Tools, a knowledge worker can define their goal in the query generator. For this goal, a state space query is generated. In CPN Tools for every possible next state, the state space query can be executed. It evaluates if this state allows reaching the goal, and more importantly, if not. This information for all next actions can be used as a recommendation to the knowledge workers (see Fig. 11).

6.2 Run-time experiments

Knowledge-intensive processes are multi-variant and often highly concurrent. Since the state space of a process grows exponentially to the degree of concurrency, model checking can be computationally expensive, slow, or even technically infeasible. To show the feasibility of our approach for the presented example, we conducted an experiment using CPN Tools: We measure the performance using the CPN Tools model checker (ASK-CTL)

⁶<http://cpntools.org/wp-content/uploads/2018/01/askctlmanual.pdf>

Create a new Objective [X]

Name
Objective 1

☐ NOT **Condition 1** ☐ type: DATA_OBJECT ☒ type: ACTIVITY

Choose the desired enabled Activity:

disburse claim [X]

☒ AND ☐ OR

☐ NOT **Condition 2** ☒ type: DATA_OBJECT ☐ type: ACTIVITY

Choose the desired Data Object State:

Advice [approve] [X]

☒ quantor: ALL lower bound 1
☐ quantor: EXISTS upper bound 1
☒ quantor: AMOUNT

☒ AND ☐ OR

☐ NOT **Condition 3** ☒ type: DATA_OBJECT ☐ type: ACTIVITY

Choose the desired Data Object State:

Assessment [approved] [X]

☒ quantor: ALL lower bound 4
☐ quantor: EXISTS upper bound
☒ quantor: AMOUNT

[+]

ABORT SAVE

Fig. 10 The user interface for specifying a goal

for our sample case model and multiple goals. All results can be found in our respective repository.⁷

Our solution requires a finite state space. To guarantee this, the model needs to be adapted so that the number of data objects is finite by require that every class is linked directly or indirectly to the case class and that all multiplicity constraints define an upper bound (* must not be used). Also, the number of concurrent fragment instances needs to be finite. All fragments require a data pre-condition to start, which is given in our example.

Our prototype needs to calculate the state space of the case model first. CPN Tools calculates the state space in 2 min 12 sec.⁸ The state space consists of 52,964 states. It needs to be calculated only once per model and can be used for all instances and all goals.

Second, we need to filter the enabled actions based on a goal. Given the following goal, ‘The knowledge worker wants to create at least four assessments, exactly one advice object, and disburse the claim afterward’, we filtered the enabled actions in 10 random states, five times each. The filtering took on average 433 ms per action. For a more complex goal, the filtering took on average 510 ms. As the state space must only be generated once for the case model, the time needed to derive recommendations in the presented example process is low enough to support knowledge workers during the execution.

⁷All results available at: <https://github.com/bptlab/fCM-query-generator>

⁸All measurements are the average of five runs on a notebook with 12 GB memory and an i7-7500U running Windows 10 and CPN Tools 4.0.1.

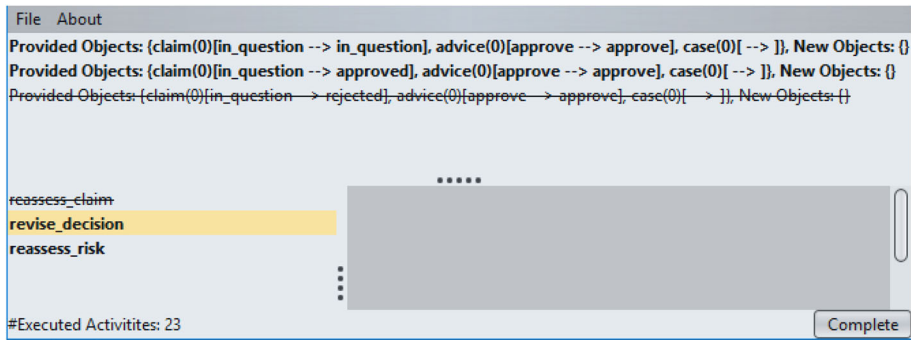


Fig. 11 Engine with recommendations: incompatible actions are struck through

6.3 Preliminary user study

To evaluate whether our approach can support the decisions of knowledge workers and drive the process to their goals, we performed an experiment.

Research question The goal of our experiments is to compare the performance of KiP executioners with or without recommendations in terms of (i) the number of attempts required, (ii) the number of actions executed, and (iii) the duration from start until reaching the goal. The hypotheses are that (i) users with recommendations need only one attempt, while users without them need more, (ii) with recommendations less actions are required to reach the goal, and (iii) with recommendations users need less time to reach the goal.

Experimental setup All 14 participants are computer science students in their bachelors (4), masters (7), or Ph.D. (3), with different but comparable experience with BPM and fCM (see the histogram in Fig. 12).

The participants were split randomly into two equally sized groups. Each participant was provided with the fCM model for our example, a predefined goal, and an execution engine.⁹ The treatment group was provided with recommendations according to their current execution state using color coding: compatible actions were colored in green, incompatible actions in red. The control group was asked to reach the goal without additional help.

Results Table 1 shows the experimental results. With recommendations, participants reached the goal in one attempt. In the control group, participants needed on avg. 1.86 attempts. The t-test for the number of attempts shows a great effect, with statistical significance for an α -value of 0.05. Hypothesis (i) holds.

When considering only the successful attempts, users that received the recommendations executed on average 19.43 actions, while users in the control group needed 20 on average. Here the effect is smaller and also shows no statistical significance. The second hypothesis cannot be proven. Still, the results indicate toward an improvement given recommendations.

⁹Engine is available at: <https://github.com/bptlab/fcm-engine>

Experience with fCM (self assessment)

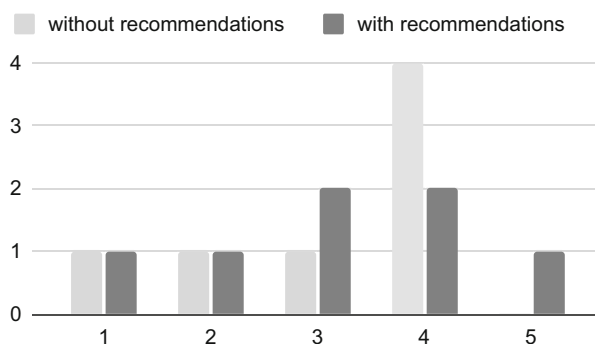


Fig. 12 Histogram for the self assessment of the participant's prior experience with fCM

When looking at how long the participants needed to reach the goal, the treatment group performed better. They needed only 422s on average, while participants without recommendations needed 634s. Also, we can observe a great statistically significant effect. The third hypothesis is supported as well.

Qualitative measurements We asked all participants “What supported you in reaching the goal,” and “What hindered you in reaching the goal”? Most participants in the control group received the most support from the case model. In contrast, most participants that received recommendations considered the red color of incompatible actions helpful. They used the model less. Participants of the control group felt hindered by the complexity of the model and wished for an improved interface for the execution engine. In contrast, out of the participants with recommendations, only two mentioned the complexity of the model as hindering. Whereas, most of them requested an improved UI as well. We assume that the recommendations reduced the need to understand the complex model fully.

Threats of validity The presented experiment yields some limitations. The number of participants was small. No participant was an actual knowledge worker. All were computer science students that are used to formal languages. In our experiments, we provided execution engines with technical interfaces. Both could be a bias.

Still, the results indicate that our approach might be helpful to support knowledge workers at run-time. However, the group of participants is too small and too homogeneous to make a general statement.

Table 1 The results of the experiments for the three hypotheses

Hypotheses	with rec.	without rec.	t-test
avg. number of attempts (i)	1.0 (sd=0)	1.86 (sd=0.9)	$t(6)=2.52, p<0.03$
avg. number of actions (ii)	19.43 (sd=1.13)	20.0 (sd=1.29)	$t(12)=0.88, p<0.20$
avg. duration (iii)	422s (sd=145)	634s (sd=143)	$t(12)=2.76, p<0.01$

7 Discussion and conclusion

We presented a model-based approach for supporting knowledge workers in deciding on the next action. They were enabled to specify goals based on an fCM model including activities and data objects. From the goal, we derive model checking problems, which can be used to filter enabled actions: For each such action, we determine whether it is compatible with the goal. This information provides valuable decision support.

Other approaches (Teinemaa et al., 2019; Francescomarino et al., 2018) propose sequences of actions. But, knowledge-intensive processes are emergent; hence, predictions are likely inaccurate, and recommending sequences of actions in advance has limited value. Our approach recommends sequences of actions in a valuable way because it iteratively considers the current situation, rather than a past situation.

We use model checking for deriving recommendations, allowing us to use established tools and techniques. However, the fCM models may have large state spaces. Experiments were conducted indicating that the presented approach provides valuable recommendations for knowledge workers.

As our approach requires explicitly modeled goals, it can be relevant to investigate the combination with existing machine learning-based approaches for computing recommendations to incorporate past behavioral patterns and tacit knowledge into decision support. Also, incremental approaches and heuristics may be applied to direct the search and cope with large state spaces. The technical feasibility of the approach in terms of run-time for computing recommendations needs to be investigated for real-world processes. A larger study should be conducted involving more diverse knowledge workers of different domains and real-world process models and changing goals. The use of eye tracking could provide relevant insights, as it is able to measure the cognitive load of knowledge workers (Wang et al., 2017; Nagel et al., 2019). Future experiments should also evaluate the presented goal modeling approach.

Overall, our approach can be applied iteratively to the execution of knowledge-intensive processes. It supports knowledge workers, who frequently have to choose from many possible actions. It is a step towards run-time support for knowledge-intensive processes. By reducing the cognitive load, the approach may enable users to execute processes faster and with fewer errors.

Funding Open Access funding enabled and organized by Projekt DEAL.

Data Availability We published the datasets generated and/or analyzed during our study and the prototypes used on Github. The fCM to CPN compiler is available at <https://github.com/bptlab/fcm2cpn>; the execution engine and the raw data from the user study at <https://github.com/bptlab/fCM-Engine/tree/experiments>; the query generator at <https://github.com/bptlab/fCM-query-generator>.

Declarations

Competing interests The authors declare that there is no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Awad, A., Weidlich, M., & Weske, M. (2011). Visually specifying compliance rules and explaining their violations for business processes. *Journal of Visual Languages and Computing*, 22(1), 30–55. <https://doi.org/10.1016/j.jvlc.2010.11.002>.
- Baier, C., & Katon, J. (2008). *Principles of model checking*. Cambridge: MIT Press.
- Ciccio, C. D., Marrella, A., & Russo, A. (2015). Knowledge-intensive processes: characteristics, requirements and analysis of contemporary approaches. *Journal on Data Semantics*, 4(1), 29–57. <https://doi.org/10.1007/s13740-014-0038-4>.
- Clarke, E. M., & Emerson, E. A. (1981). Design and synthesis of synchronization skeletons using branching-time temporal logic. In *Logics of Programs, Workshop, Yorktown Heights, New York, USA, May 1981. Lecture Notes in Computer Science*, (Vol. 131 pp. 52–71). Berlin: Springer, <https://doi.org/10.1007/BFb0025774>.
- Clarke, E. M., Grumberg, O., & Peled, D.A. (2001). *Model Checking*. MIT Press. <https://mitpress.mit.edu/books/model-checking>.
- Elliott, R. K., & Jacobson, P. D. (2002). The evolution of the knowledge professional. *Accounting Horizons*, 16(1), 69–80. <https://doi.org/10.2308/acch.2002.16.1.69>.
- Estañol, M. (2017). Artifact-centric business process models in UML: specification and reasoning. In *Proceedings of the BPM Demo Track and BPM Dissertation Award Co-located with 15th International Conference on Business Process Modeling (BPM 2017), Barcelona, Spain, September 13, 2017. CEUR Workshop Proceedings*, vol. 1920. CEUR-WS.org.
- Francescomarino, C. D., Ghidini, C., Maggi, F. M., & Milani, F. (2018). Predictive process monitoring methods: Which one suits me best? In *Business Process Management - 16th International Conference, BPM 2018, Proceedings. Lecture Notes in Computer Science*, (Vol. 11080 pp. 462–479). Cham: Springer, https://doi.org/10.1007/978-3-319-98648-7_27.
- Galrinho, L., Seco, J. C., Debois, S., Hildebrandt, T., Norman, H., & Slaats, T. (2021). Regrada: Reactive graph data. In *International Conference on Coordination Languages and Models*, pp. 188–205. Springer.
- Haarmann, S. (2021). Fragment-based case management models: Metamodel, consistency, & correctness. In *Proceedings of the 13th European Workshop on Services and Their Composition (ZEUS 2021). CEUR Workshop Proceedings*, (Vol. 2839 pp. 1–8). Aachen: CEUR-WS.org.
- Haarmann, S., Montali, M., & Weske, M. (2021). Refining case models using cardinality constraints. In M. L. Rosa, S. W. Sadiq, & E. Teniente (Eds.) *Advanced Information Systems Engineering - 33rd International Conference, CAiSE 2021, Proceedings. Lecture Notes in Computer Science*, (Vol. 12751 pp. 296–310). Cham: Springer, https://doi.org/10.1007/978-3-030-79382-1_18.
- Haarmann, S., Seidel, A., & Weske, M. (2021). Modeling objectives of knowledge workers. In *Business Process Management Workshops - BPM 2021 International Workshops, Revised Selected Papers. Lecture Notes in Business Information Processing*, (Vol. 436 pp. 337–348). Cham: Springer, https://doi.org/10.1007/978-3-030-94343-1_26.
- Haarmann, S., & Weske, M. (2020). Correlating data objects in fragment-based case management. In *Business Information Systems - 23rd International Conference, BIS 2020, Proceedings. Lecture Notes in Business Information Processing*, (Vol. 389 pp. 197–209). Cham: Springer, https://doi.org/10.1007/978-3-030-53337-3_15.
- Hewelt, M., & Weske, M. (2016). A hybrid approach for flexible case modeling and execution. In *Business Process Management Forum - BPM Forum 2016, Proceedings. Lecture Notes in Business Information Processing*, (Vol. 260 pp. 38–54). Cham: Springer, https://doi.org/10.1007/978-3-319-45468-9_3.
- Hildebrandt, T. T., & Mukkamala, R. R. (2010). Declarative event-based workflow as distributed dynamic condition response graphs. In *Proceedings Third Workshop on Programming Language Approaches to Concurrency and communication-cEntric Software, PLACES 2010*, (Vol. 69 pp. 59–73). EPTCS, <https://doi.org/10.4204/EPTCS.69.5>.
- Hull, R., Damaggio, E., Fournier, F., Gupta, M., Heath, F. F. T. III., Hobson, S., Linehan, M. H., Maradugu, S., Nigam, A., Sukaviriya, P., & Vuculín, R. (2010). Introducing the guard-stage-milestone approach for specifying business entity lifecycles. In *Web Services and Formal Methods - 7th International Workshop, WS-FM 2010. Revised Selected Papers. Lecture Notes in Computer Science*, (Vol. 6551 pp. 1–24). Berlin: Springer, https://doi.org/10.1007/978-3-642-19589-1_1.
- Knuplesch, D., & Reichert, M. (2017). A visual language for modeling multiple perspectives of business process compliance rules. *Software and Systems Modeling*, 16(3), 715–736. <https://doi.org/10.1007/s10270-016-0526-0>.

- Künzle, V., & Reichert, M. (2011). Philharmonicflows: towards a framework for object-aware process management. *Journal of Software Maintenance and Evolution: Research and Practice*, 23(4), 205–244. <https://doi.org/10.1002/smr.524>.
- Marrella, A., Mecella, M., Sardiña, S., & Tucceri, P. (2014). Smartpm: Automated adaptation of dynamic processes. In *Service-oriented Computing - ICSOC 2014 Workshops - WESOA; semaPS, RMSOC, KASA, ISC, FOR-MOVES, CCSA and Satellite Events, Paris, France, Revised Selected Papers. Lecture Notes in Computer Science*, (Vol. 8954 pp. 423–427). Cham: Springer, https://doi.org/10.1007/978-3-319-22885-3_40.
- Meyer, A., Herzberg, N., Puhlmann, F., & Weske, M. (2014). Implementation framework for production case management: Modeling and execution. In *18th IEEE International Enterprise Distributed Object Computing Conference, EDOC 2014* (pp. 190–199). New York: IEEE Computer Society, <https://doi.org/10.1109/EDOC.2014.34>.
- Nagel, S., Corea, C., & Delfmann, P. (2019). Effects of quantitative measures on understanding inconsistencies in business rules. In *52Nd Hawaii international conference on system sciences, HICSS 2019*, pp. 1–10.
- Object Management Group (2014). Business Process Model and Notation (BPMN). <https://www.omg.org/spec/BPMN>.
- Object Management Group (2016). Case Management Model and Notation (CMMN). <https://www.omg.org/spec/CMMN>.
- Object Management Group (2017). Unified Modeling Language (UML). <https://www.omg.org/spec/UML>.
- Pesic, M., Schonenberg, H., & van der Aalst, W. M. P. (2007). DECLARE: Full support for loosely-structured processes. In *11th IEEE International Enterprise Distributed Object Computing Conference (EDOC 2007)* (pp. 287–300). New York: IEEE Computer Society, <https://doi.org/10.1109/EDOC.2007.14>.
- Pyöriä, P. (2005). The concept of knowledge work revisited. *Journal of Knowledge Management*, 9(3), 116–127. <https://doi.org/10.1108/13673270510602818>.
- Rozinat, A., Wynn, M. T., van der Aalst, W. M. P., ter Hofstede, A. H. M., & Fidge, C. J. (2009). Workflow simulation for operational decision support. *Data & Knowledge Engineering*, 68(9), 834–850. <https://doi.org/10.1016/j.datak.2009.02.014>.
- Seco, J. C., Debois, S., Hildebrandt, T., & Slaats, T. (2018). Reseda: declaring live event-driven computations as reactive semi-structured data. In *2018 IEEE 22nd International Enterprise Distributed Object Computing Conference (EDOC)*, (pp. 75–84) IEEE.
- Snoeck, M. (2014). Enterprise Information Systems Engineering - The MERODE Approach. The Enterprise Engineering Series, Springer, Cham. <https://doi.org/10.1007/978-3-319-10145-3>.
- Snoeck, M., Smedt, J. D., & Weerdt, J. D. (2021). Supporting data-aware processes with MERODE. In *Enterprise, Business-process and Information Systems Modeling - 22nd International Conference, BPMDS 2021, and 26th International Conference, EMMSAD 2021, Held at CAiSE 2021, Proceedings. Lecture Notes in Business Information Processing*, (Vol. 421 pp. 131–146). Cham: Springer, https://doi.org/10.1007/978-3-030-79186-5_9.
- Sprovieri, D., & Vogler, S. (2015). Run-time composition of partly structured business processes using heuristic planning. In *International Conference on Enterprise Systems, ES 2015* (pp. 225–232). New York: IEEE, <https://doi.org/10.1109/ES.2015.30>.
- Steinau, S., Marrella, A., Andrews, K., Leotta, F., Mecella, M., & Reichert, M. (2019). DALEC: A framework for the systematic evaluation of data-centric approaches to process management software. *Software and Systems Modeling*, 18(4), 2679–2716. <https://doi.org/10.1007/s10270-018-0695-0>.
- Swenson, K. D. (2012). Position: BPMN is incompatible with ACM. In *Business Process Management Workshops - BPM 2012 International Workshops. Revised Papers. Lecture Notes in Business Information Processing*, (Vol. 132 pp. 55–58). Berlin: Springer, https://doi.org/10.1007/978-3-642-36285-9_7.
- Swenson, K. (2013). State of the art in case management. http://kswenson.purplehillsbooks.com/2013/State-of-the-Art-In-Case-Management_2013.pdf.
- Teinemaa, I., Dumas, M., Rosa, M. L., & Maggi, F. M. (2019). Outcome-oriented predictive process monitoring: Review and benchmark. *ACM Transactions on Knowledge Discovery from Data*, 13(2), 17–11757. <https://doi.org/10.1145/3301300>.
- Venero, S. K., Schmerl, B. R., Montecchi, L., dos Reis, J., & Rubira, C. M. F. (2020). Automated planning for supporting knowledge-intensive processes. In *Enterprise, Business-process and Information Systems Modeling - 21st International Conference, BPMDS 2020, 25th International Conference, EMMSAD 2020, Held at CAiSE 2020, Proceedings. Lecture Notes in Business Information Processing*, (Vol. 387 pp. 101–116). Cham: Springer, https://doi.org/10.1007/978-3-030-49418-6_7.
- Wang, W., Indulska, M., Sadiq, S. W., & Weber, B. (2017). Effect of linked rules on business process model understanding. In *Business process management - 15th international conference, BPM*

- 2017, *proceedings. Lecture notes in computer science*, (Vol. 10445 pp. 200–215). Cham: Springer, https://doi.org/10.1007/978-3-319-65000-5_12.
- Weske, M. (2019). *Business Process Management - Concepts, Languages, Architectures*, 3rd. Berlin: Springer.
- Wynn, M. T., Dumas, M., Fidge, C. J., ter Hofstede, A. H. M., & van der Aalst, W. M. P. (2007). Business process simulation for operational decision support. In *Business Process Management Workshops, BPM 2007 International Workshops, BPI, BPD, CBP, ProHealth, RefMod, Semantics4ws, Revised Selected Papers. Lecture Notes in Computer Science*, (Vol. 4928 pp. 66–77). Berlin: Springer, https://doi.org/10.1007/978-3-540-78238-4_8.
- van der Aalst, W. M., Artale, A., Montali, M., & Tritini, S. (2017). Object-centric behavioral constraints: Integrating data and declarative process modelling. *Description Logics*, 1879.
- van der Aalst, W. M. P., Pesic, M., & Schonenberg, H. (2009). Declarative workflows: Balancing between flexibility and support. *Computer Science - Research and Development*, 23(2), 99–113. <https://doi.org/10.1007/s00450-009-0057-9>.
- van der Aalst, W. M. P., van Hee, K. M., ter Hofstede, A. H. M., Sidorova, N., Verbeek, H. M. W., Voorhoeve, M., & Wynn, M. T. (2011). Soundness of workflow nets: classification, decidability, and analysis. *Formal Aspects Computing*, 23(3), 333–363. <https://doi.org/10.1007/s00165-010-0161-4>.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.